Detection APC 注入的蓝队检测视角

前两章我们完成了 APC 注入 和 Early Bird：用 NtQueueApcThread 把 Shellcode 挂到目标线程的 APC 队列，等线程进入可提醒状态时执行。它绕过了 CreateRemoteThread 的监控，但不是隐形。

本章我们站在蓝队（EDR）立场，分析 APC 注入和 Early Bird 的可检测点，并给出反制策略。

- **本章目标**

理解 EDR 检测 APC 注入的两种核心手段：

- 监控 NtQueueApcThread 调用（正常程序极少跨进程插入 APC）。
- 监控带 CREATE_SUSPENDED 标志的进程创建（Early Bird 的入口特征）。

掌握至少两种反制策略：

- 用 NtCreateThreadEx 的隐藏标志替代 APC。
- 用 SetThreadContext 替代 APC。

- **为什么 APC 注入是可疑的**

正常程序几乎从不跨进程插入 APC。APC 机制的设计初衷是“同个进程内的线程间通信”或“内核 I/O 完成通知”。一个进程往另一个进程的线程里插 APC，本身就是异常行为。

蓝队关注的核心问题：

- 谁在调用 NtQueueApcThread?
- 目标线程属于哪个进程？和调用者是不是同一个进程?
- 插入的 APC 回调地址落在哪里?
- 这个进程是不是刚刚以 CREATE_SUSPENDED 启动的?

- **检测手段一：监控 NtQueueApcThread 调用**

(一) EDR 如何监控

EDR 通过以下三种方式监控 APC 插入行为:

监控方式	物理动作
用户态 Hook	在 ntdll.dll!NtQueueApcThread 入口处写入跳转指令，拦截所有调用
内核回调	注册 PsSetCreateThreadNotifyRoutine 等回调，或者通过 ETW 订阅 APC 相关事件
ETW 事件	启用 MicrosoftWindowsKernelAuditAPICalls 提供程序，记录 NtQueueApcThread 调用

(二) 检测逻辑

EDR 收到 NtQueueApcThread 调用时，会检查：

- 调用者进程 ID 和 目标线程所属进程 ID 是否相同?
  - ✓ 相同 → 进程内 APC，正常。
  - ✓ 不同 → 跨进程 APC，高危。
- APC 回调地址 是否落在合法模块的可执行段内?
  - ✓ 落在 ntdll.dll、kernel32.dll 等已知模块内 → 相对正常。
  - ✓ 落在未知内存、VirtualAllocEx 申请的内存、或.text 节被修改的区域 → 高危。
- 调用频率 是否异常?
  - ✓ 短时间内大量插入 APC → 可疑。

我们的 APC 注入代码：

```
NtQueueApcThread(hThread, (PVOID)pRemoteMem, NULL, NULL, NULL);
// 调用者：我们的注入器进程
// 目标线程：notepad.exe 的线程
// 回调地址：pRemoteMem（VirtualAllocEx 申请的新内存）
```

EDR 会看到：“进程 A 向进程 B 的线程插入了个 APC，回调地址指向进程 B 里一块新申请的可执行内存。” → 三条特征全部命中，直接告警。

- **检测手段二：监控 CREATE_SUSPENDED 进程创建**

(一) 为什么 Early Bird 会被盯上

Early Bird 的第一步是：

```
CreateProcessA(NULL, "notepad.exe", ..., CREATE_SUSPENDED, ...);
```

CREATE_SUSPENDED 本身不是恶意行为——很多正常程序（比如调试器、安装程序）也会用挂起方式创建进程。但结合后续行为，它就成了特征：

- 创建一个挂起进程。
- 在进程还没开始运行时，就往里写内存、插 APC。
- 然后恢复线程。

这个“创建挂起 → 写内存 → 插 APC → 恢复”的时序链，是 Early Bird 的经典特征。

(二) EDR 如何监控

监控方式	物理动作
进程创建回调	注册 PsSetCreateProcessNotifyRoutineEx，在进程创建时拿到 dwCreationFlags
ETW 进程事件	订阅 MicrosoftWindowsKernelProcess，记录进程创建时的标志位
行为关联分析	记录“挂起创建的进程”，然后监控它后续是否被 WriteProcessMemory、NtQueueApcThread 操作

(三) 检测逻辑

EDR 会建立条“行为链”：

```
进程 A 以 CREATE_SUSPENDED 创建进程 B
    ↓
进程 A 对进程 B 调用 VirtualAllocEx / WriteProcessMemory
    ↓
进程 A 对进程 B 的主线程调用 NtQueueApcThread
    ↓
进程 A 对进程 B 的主线程调用 ResumeThread
    ↓
【告警】完整的 Early Bird 注入链
```

单独看每步都不是恶意的，但连起来就是攻击链。这就是 EDR 的“行为关联分析”能力。

- **检测手段三：APC 队列异常与调用栈分析**

(一) APC 队列异常

EDR 可以定期扫描目标进程的线程 APC 队列（通过内核回调或内存扫描），检查：

- APC 队列里是否有指向“非模块内存”的回调。
- APC 回调地址是否和磁盘上的模块字节一致。

(二) 调用栈分析

当 APC 被执行时，EDR 可以通过 ETW 记录执行时的调用栈：

- 如果调用栈里出现“未知模块”或“新申请内存”，可疑。
- 如果 APC 执行时间点恰好是进程启动初期（Early Bird），更可疑。

- **反制策略一：用 NtCreateThreadEx 的隐藏标志**

(一) 为什么 NtCreateThreadEx 比 CreateRemoteThread 隐蔽

CreateRemoteThread 是 kernel32.dll 导出的 API，EDR 重点监控它。NtCreateThreadEx 是 ntdll.dll 导出的 NativeAPI，更底层，Hook 难度更高。

(二) NtCreateThreadEx 的“隐藏标志”

NtCreateThreadEx 的 Flags 参数支持一些特殊标志，可以改变线程创建行为：

标志	物理意义
THREAD_CREATE_FLAGS_HIDE_FROM_DEBUGGER	线程对调试器隐藏
THREAD_CREATE_FLAGS_CREATE_SUSPENDED	创建挂起线程
THREAD_CREATE_FLAGS_SKIP_THREAD_ATTACH	跳过 DLL 的线程附加通知

关键：SKIP_THREAD_ATTACH 可以跳过 EDR DLL 的 DLL_THREAD_ATTACH 通知，让 EDR 少一次感知机会。

(三) 反制效果

用 NtCreateThreadEx 替代 CreateRemoteThread，可以：

- 避开 CreateRemoteThread 的用户态 Hook。
- 用隐藏标志减少 EDR 的感知点。

注意：这仍是用户态 API，如果 EDR Hook 了 ntdll.dll!NtCreateThreadEx，依然会被发现。

- **反制策略二：用 SetThreadContext 替代 APC**

(一) 什么是 SetThreadContext

SetThreadContext 可以修改线程的寄存器上下文（包括 RIP，即下条要执行的指令地址）。

物理动作：把目标线程的 RIP 改成 Shellcode 的地址，然后恢复线程，线程就会从 Shellcode 开始执行。

(二) 流程

- 创建挂起进程（CREATE_SUSPENDED）
- 写入 Shellcode
- 用 GetThreadContext 拿到主线程的当前上下文
- 修改上下文中的 RIP 为 Shellcode 地址
- 用 SetThreadContext 写回
- ResumeThread

(三) 为什么能绕过 APC 监控

- 不调用 NtQueueApcThread → APC 监控点完全失效。
- 不创建新线程 → 远程线程监控点失效。

- 执行点仍然是合法进程的主线程。

(四) 检测点

SetThreadContext 本身也会被 EDR 监控。但它的检测逻辑和 APC 不同：

- EDR 检查“谁在修改其他进程的线程上下文”。
- 如果修改的是 RIP，且新 RIP 指向非模块内存 → 高危。

反制：把 Shellcode 写进合法模块的 .text 节（Module Stomping），让新 RIP 指向合法模块 → 降低检测等级。

- **三种技术的检测对比**

技术	核心 API	EDR 主要监控点	检测难度
CreateRemoteThread	CreateRemoteThread	远程线程创建	低
普通 APC	NtQueueApcThread	跨进程 APC 插入	中
Early Bird	CREATE_SUSPENDED +NtQueueApcThread	挂起创建+APC 插入行为链	中高
SetThreadContext	SetThreadContext	跨进程线程上下文修改	中
直接系统调用	syscall 指令	内核层 ETW / 回调	高

- **蓝队检测的最终逻辑**

检测点	蓝队看什么	红队反制
跨进程 APC	调用者和目标线程是否同一进程	用 SetThreadContext 替代 APC
APC 回调地址	是否落在合法模块内	用 Module Stomping 把 Shellcode 写进合法模块
CREATE_SUSPENDED	挂起创建后是否立即写内存、插 APC	延迟操作、或用其他进程创建方式
行为链关联	挂起创建 → 写内存 → 插 APC → 恢复	拆分操作时序、插入无关操作干扰分析
线程上下文修改	SetThreadContext 是否修改了 RIP	让新 RIP 指向合法模块

- **小结**
  - APC 注入的可检测性高于 CreateRemoteThread，但不是零。
  - 最大暴露：跨进程 NtQueueApcThread 调用 + APC 回调地址不在合法模块内。
  - Early Bird 的暴露点：CREATE_SUSPENDED+后续写内存+插 APC 的完整行为链。
  - 反制核心：用 SetThreadContext 替代 APC，用 NtCreateThreadEx 的隐藏标志。